

Universidad de la Fuerzas Armadas - ESPE
Departamento de Ciencias de la Computación
Carrera de Ingeniería de Software



Análisis Y Diseño de Software - NRC: 30723

Guías 1 y 2 del CRUD de estudiante

Grupo: 4

Tufiño Erick

Santi Jeancarlo

Cedeño Andrés

Profesor: Ing. Jenny Ruiz

Fecha: 28 de Abril de 2026

1. Casos de uso

1.1. Requisitos Funcionales de nivel 0

- RF-01: El sistema debe permitir agregar un estudiante con ID, Nombre y Edad.
- RF-02: El sistema debe permitir actualizar los datos de un estudiante existente.
- RF-03: El sistema debe permitir eliminar un estudiante existente cuando corresponda.
- RF-04: El sistema debe permitir mostrar todos los estudiantes registrados.
- RF-05: El sistema debe validar los datos obligatorios antes de agregar o actualizar.

1.2. Especificación

Actor Principal: Administrador académico.

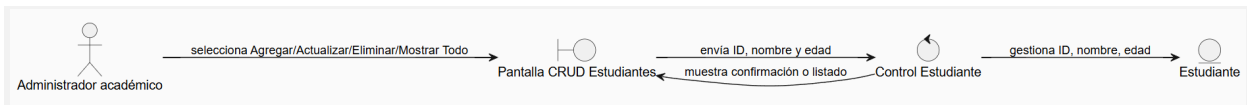
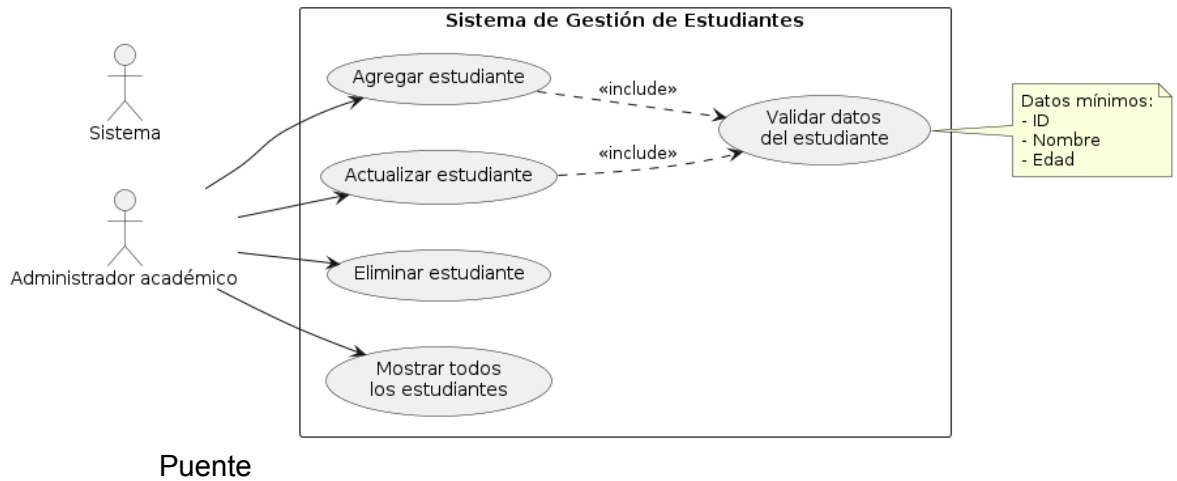
Agregar estudiante: Ingresar ID, Nombre y Edad -> validar datos -> verificar ID único -> guardar estudiante. (Resultado: Estudiante agregado y visible en tabla).

Actualizar estudiante: Ingresar ID y datos nuevos -> validar datos -> buscar estudiante -> guardar cambios. (Resultado: Datos actualizados).

Eliminar estudiante: Seleccionar/ingresar ID -> buscar estudiante -> confirmar y eliminar. (Resultado: Estudiante retirado de la tabla).

Mostrar todos los estudiantes: Solicitar listado -> recuperar registros -> mostrar tabla. (Resultado: Tabla actualizada con registros).

1.3. Diagrama de Casos de Uso



Código PlantUML

```

@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
actor "Administrador académico" as Admin
actor "Sistema" as Sistema

rectangle "Sistema de Gestión de Estudiantes" {
    usecase "Agregar estudiante" as UC1
    usecase "Actualizar estudiante" as UC2
    usecase "Eliminar estudiante" as UC3
    usecase "Mostrar todos estudiantes" as UC4
    usecase "Validar datos estudiante" as UC5
}

Admin -- UC1
Admin -- UC2
Admin -- UC3
Admin -- UC4
UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>
note right of UC5
    Datos mínimos:
    - ID
    - Nombre
    - Edad
end note
@enduml

```

2. Clases de análisis

2.1. Responsabilidades Identificadas

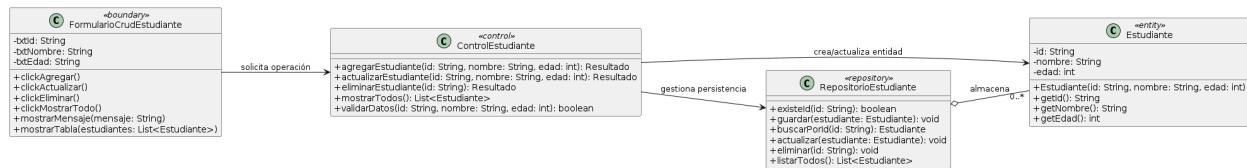
Frontera (Boundary) - FormularioCrudEstudiante: Representa la pantalla con los campos (ID, Nombre, Edad) y botones. Recibe datos y muestra resultados.

Control - ControlEstudiante: Coordina la lógica del caso de uso. Valida los datos y decide las acciones sobre el repositorio.

Entidad (Entity) - Estudiante: Representa la información del dominio, conteniendo los atributos id, nombre y edad.

Repositorio - RepositorioEstudiante: Representa la colección/persistencia que permite guardar, buscar, actualizar, eliminar y listar.

2.2. Diagrama de Clases de Análisis



Código PlantUML

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class FormularioCrudEstudiante <<boundary>> {
    - txtId : String
    - txtNombre : String
    - txtEdad : String
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
    + mostrarMensaje(mensaje: String)
    + mostrarTabla(estudiantes: List<Estudiante>)
}

class ControlEstudiante <<control>> {
    + agregarEstudiante(id: String, nombre: String, edad: int): Resultado
    + actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
    + eliminarEstudiante(id: String): Resultado
    + mostrarTodos(): List<Estudiante>
    + validarDatos(id: String, nombre: String, edad: int): boolean
}

class RepositorioEstudiante <<repository>> {
    + existeId(id: String): boolean
    + guardar(estudiante: Estudiante): void
    + buscarPorId(id: String): Estudiante
    + actualizar(estudiante: Estudiante): void
    + eliminar(id: String): void
    + listarTodos(): List<Estudiante>
}

class Estudiante <<entity>> {
    - id: String
    - nombre: String
    - edad: int
    + Estudiante(id: String, nombre: String, edad: int)
    + getId(): String
    + getNombre(): String
    + getEdad(): int
}

FormularioCrudEstudiante --> ControlEstudiante : solicita operación
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante --> Estudiante : crea/actualiza entidad
RepositorioEstudiante --> Estudiante : almacena
```

```
+ actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
+ eliminarEstudiante(id: String): Resultado
+ mostrarTodos(): List<Estudiante>
- validarDatos(id: String, nombre: String, edad: int): boolean
}
```

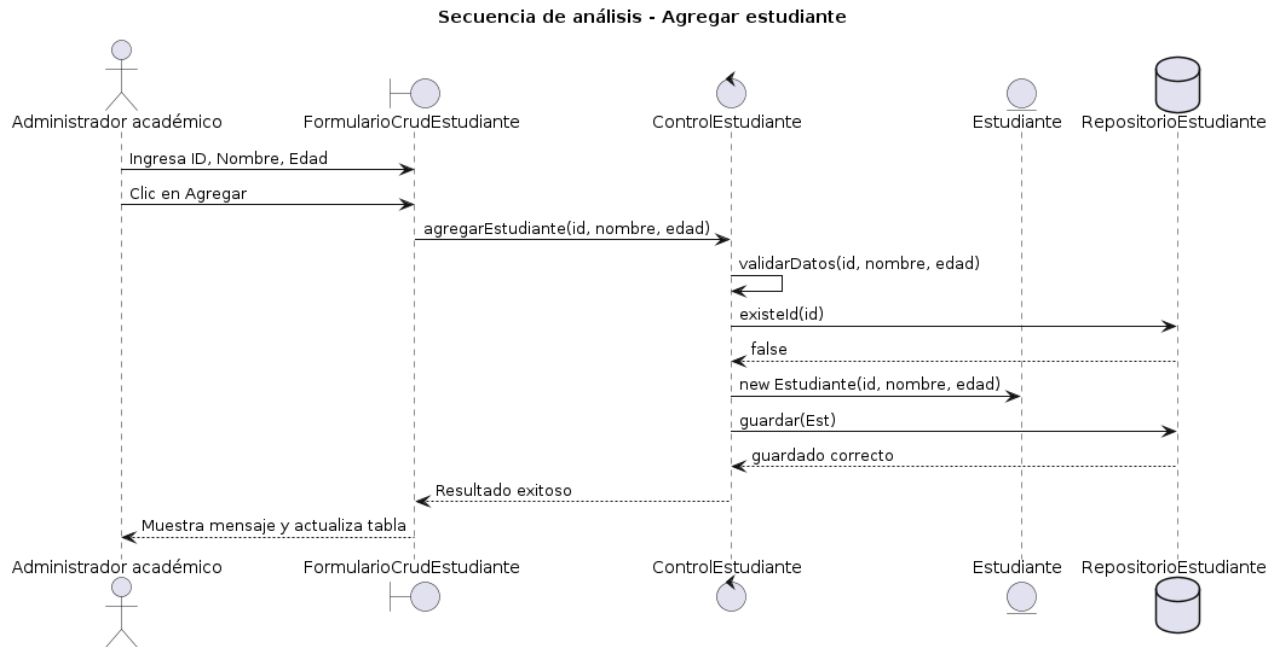
```
class Estudiante <<entity>> {
- id : String
- nombre : String
- edad : int
+ Estudiante(id: String, nombre: String, edad: int)
+ getId(): String
+ getNombre(): String
+ getEdad(): int
}
```

```
class RepositorioEstudiante <<repository>> {
+ existeId(id: String): boolean
+ guardar(estudiante: Estudiante): void
+ buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
+ eliminar(id: String): void
+ listarTodos(): List<Estudiante>
}
```

```
FormularioCrudEstudiante -- ControlEstudiante : solicita operación >
ControlEstudiante -- Estudiante : crea/actualiza entidad >
ControlEstudiante -- RepositorioEstudiante : gestiona persistencia >
RepositorioEstudiante o-- "0..*" Estudiante : almacena >
@enduml
```

3. Secuencias de análisis

3.1. Secuencia: Agregar estudiante

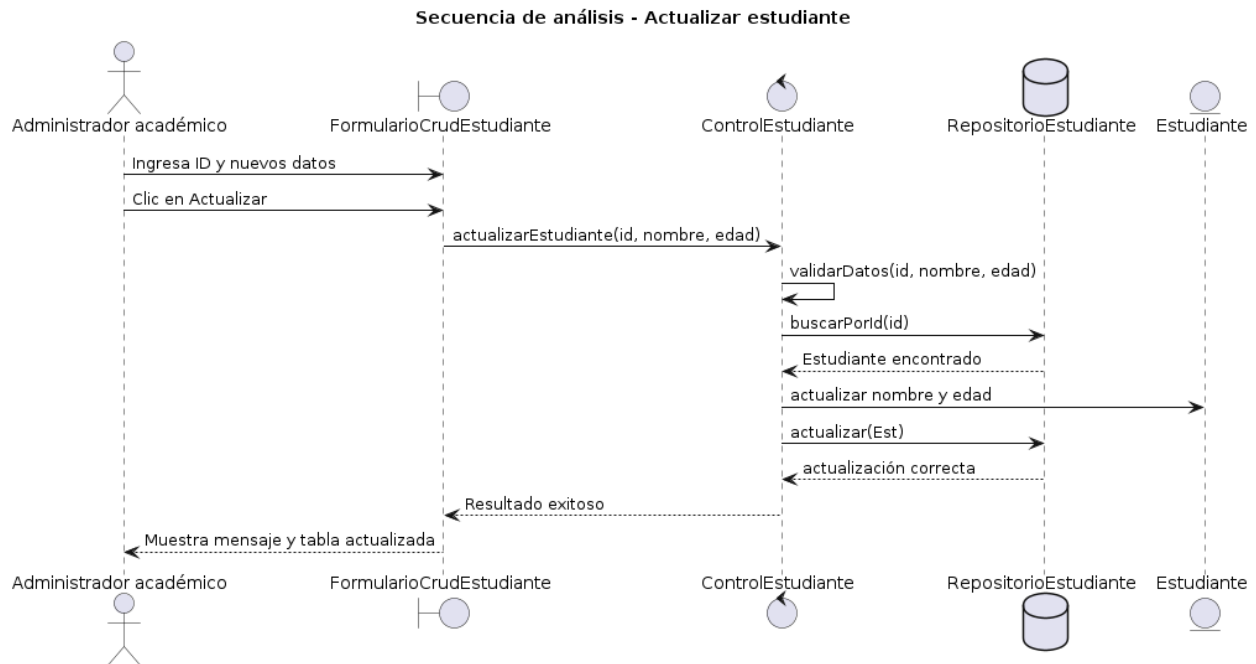


Código PlantUML

```
@startuml
title Secuencia de análisis - Agregar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
entity "Estudiante" as Est
database "RepositorioEstudiante" as Repo

Admin -> UI : Ingresar ID, Nombre, Edad
Admin -> UI : Clic en Agregar
UI -> Ctrl : agregarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : existeId(id)
Repo --> Ctrl : false
Ctrl -> Est : new Estudiante(id, nombre, edad)
Ctrl -> Repo : guardar(Est)
Repo --> Ctrl : guardado correcto
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```

3.2. Secuencia: Actualizar estudiante



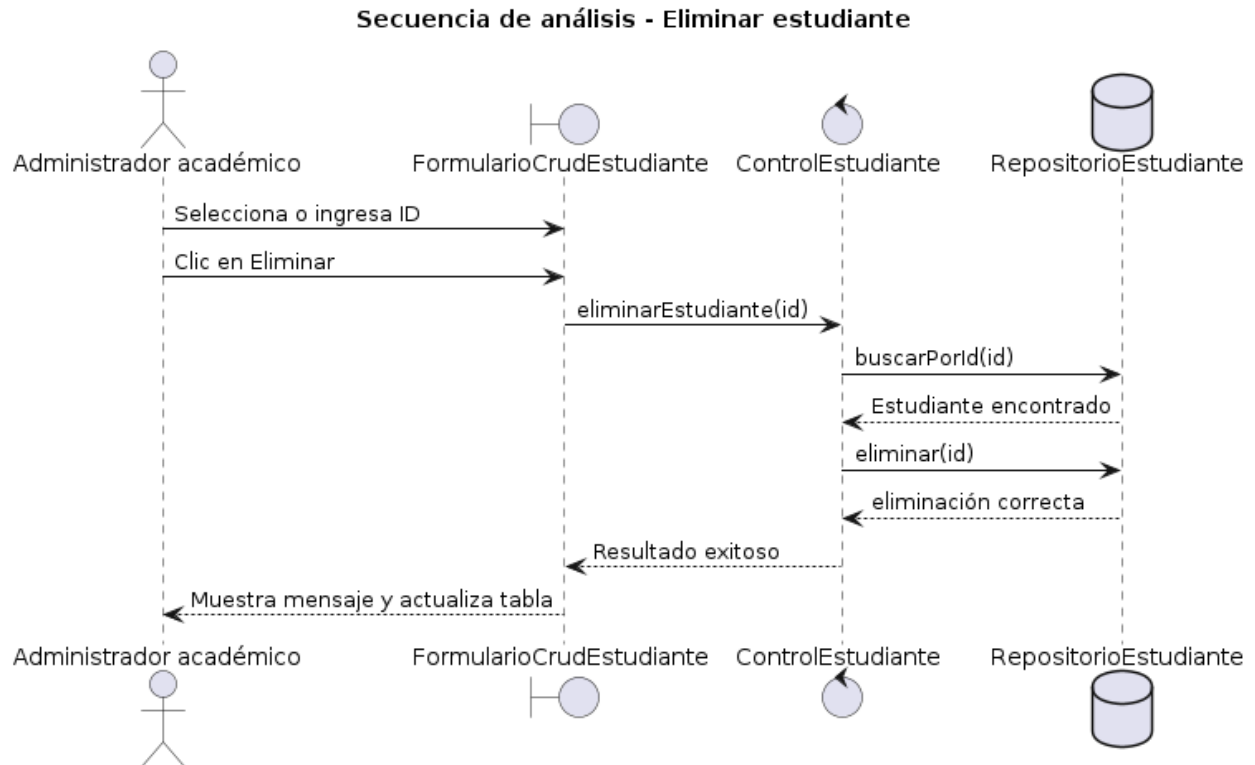
Código PlantUML

```

@startuml
title Secuencia de análisis - Actualizar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo
entity "Estudiante" as Est

Admin -> UI : Ingresa ID y nuevos datos
Admin -> UI : Clic en Actualizar
UI -> Ctrl : actualizarEstudiante(id, nombre, edad)
Ctrl -> Ctrl : validarDatos(id, nombre, edad)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Est : actualizar nombre y edad
Ctrl -> Repo : actualizar(Est)
Repo --> Ctrl : actualización correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y tabla actualizada
@enduml
  
```

3.3. Secuencia: Eliminar estudiante



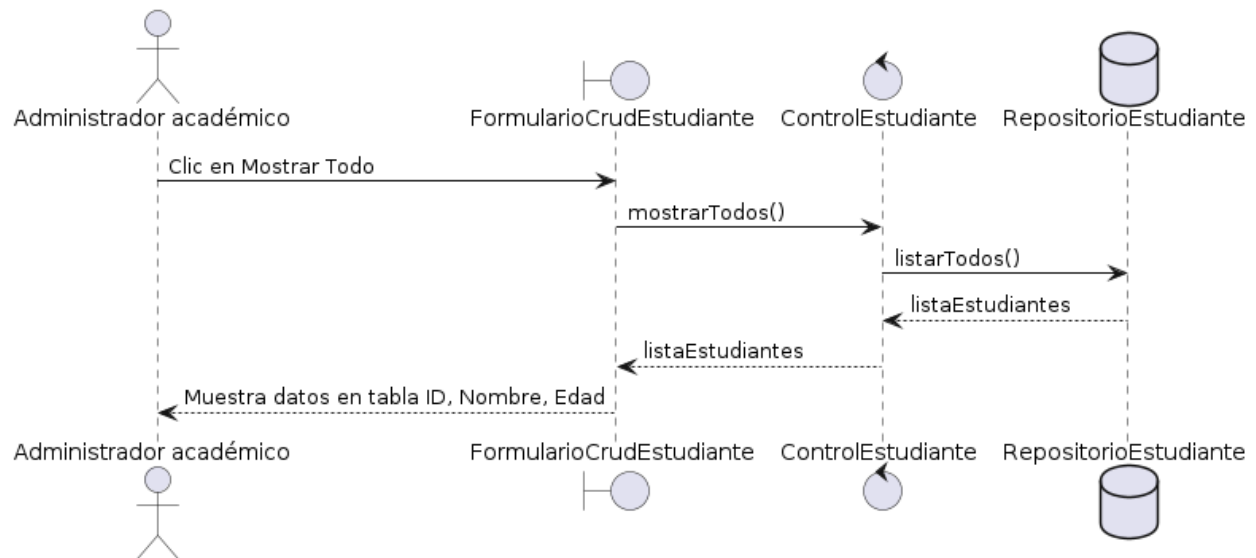
Código PlantUML

```
@startuml
title Secuencia de análisis - Eliminar estudiante
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Selecciona o ingresa ID
Admin -> UI : Clic en Eliminar
UI -> Ctrl : eliminarEstudiante(id)
Ctrl -> Repo : buscarPorId(id)
Repo --> Ctrl : Estudiante encontrado
Ctrl -> Repo : eliminar(id)
Repo --> Ctrl : eliminación correcta
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje y actualiza tabla
@enduml
```


3.4. Secuencia: Mostrar todos los estudiantes

Secuencia de análisis - Mostrar todos los estudiantes



Código PlantUML

```
@startuml
title Secuencia de análisis - Mostrar todos los estudiantes
actor "Administrador académico" as Admin
boundary "FormularioCrudEstudiante" as UI
control "ControlEstudiante" as Ctrl
database "RepositorioEstudiante" as Repo

Admin -> UI : Clic en Mostrar Todo
UI -> Ctrl : mostrarTodos()
Ctrl -> Repo : listarTodos()
Repo --> Ctrl : listaEstudiantes
Ctrl --> UI : listaEstudiantes
UI --> Admin : Muestra datos en tabla (ID, Nombre, Edad)
@enduml
```

4. Trazabilidad

Matriz de relación integral:

Requisito Funcional (RF)	Caso de Uso	Clases Identificadas (Boundary, Control, Entity, Repo)	Secuencia de Análisis Correspondiente
RF-01: Agregar estudiante y RF-05: Validar datos.	Agregar estudiante	FormularioCrudEstudiante, ControlEstudiante, Estudiante, RepositorioEstudiante	Secuencia de análisis - Agregar estudiante
RF-02: Actualizar datos y RF-05: Validar datos.	Actualizar estudiante	FormularioCrudEstudiante, ControlEstudiante, Estudiante, RepositorioEstudiante	Secuencia de análisis - Actualizar estudiante
RF-03: Eliminar estudiante.	Eliminar estudiante	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Secuencia de análisis - Eliminar estudiante
RF-04: Mostrar estudiantes.	Mostrar todos los estudiantes	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Secuencia de análisis - Mostrar todos los estudiantes

5. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia demuestra que el modelado de software mantiene coherencia cuando cada elemento cumple su responsabilidad: los casos de uso definen qué requiere el actor, las clases establecen quién lo resuelve y las secuencias muestran cómo colaboran los objetos. Esta coherencia se confirmó al implementar el código Java del sistema, donde cada clase del análisis (FormularioCrudEstudiante, ControlEstudiante, Estudiante, RepositorioEstudiante) se tradujo directamente al código con los mismos métodos definidos en los diagramas, y flujos como la validación previa al guardado o la verificación de ID duplicado coincidieron con la lógica implementada. Se concluye que un modelo de análisis bien construido no solo clarifica el diseño, sino que actúa como guía directa para la implementación.